This was just a very concise and compressed taste of potential observability use cases. If you would like to learn more about observability use cases, signals, and methods, I strongly encourage you to peruse the "Observability Whitepaper" (which I had the pleasure of contributing to; http://mng.bz/ EQJO) by the Cloud Native Computing Foundation's Technical Advisory Group for Observability or, more informally, *CNCF TAG o11y*. (The term *o11y* refers to *observability*).

Observability vs. monitoring

Is *observability* just a new buzzword for *monitoring*, or are these distinct concepts that relate to each other? When monitoring, you're looking at the health of a system (e.g., your platform or a particular service). Observability is conceptually broader; while health is still relevant, it also considers changes in your system, such as what the impact of a code change is or what external dependencies contribute to (for example, the p90 of the response time of one of your services). Marketing terms aside, you can think of monitoring as shorthand for *system health and performance-related observability*, and that's how I will treat it in this book.

With some use cases under our belts, let's move on to different roles within an organization. We'll discuss their goals and tasks to more fully appreciate and understand how observability can assist different people on your team and in the wider organization.

1.3 Roles and goals

Nowadays, building products is a team sport. Except in the case of a small tool, a variety of different job functions or roles typically work together to realize a software-based offering. The different roles involved in the software creation and operation process focus on different tasks along the application life cycle:

- *Developers*—Developers write code, test and document it, and make artifacts, such as a container image, a binary, or a script (for interpreted languages, such as Python), available for deployment. Observability can help to more quickly iterate (finding bugs via logs and traces), optimize runtime behavior (identifying slow code paths via traces), and support operations (via metrics and logs).
- *Release and test engineers*—A release engineer can own CI and/or CD pipelines and needs visibility into build, test, and deploy processes. In this context, some organizations have a dedicated tester (or QA tester) role that has similar requirements as those for release engineers.
- *DevOps roles*—Some teams have DevOps roles, which are partially responsible for code, often own operations (are on call), and establish and enforce good practices. Similar considerations as with developers apply concerning observability, and in addition, metrics are typically useful for either dashboards or alerts. One notable DevOps role is an SRE, who coaches developers or supports other DevOps roles, helping them to meet certain operational goals (we will come back to that in the last chapter on SLOs and SLIs).